# Audit and Validation Report

**Project 3: PowerShell Automation for Active Directory**
**Author:** Mohd. Ismayeel Khan

**Date:** 14 October 2026

**Environment:** VMware Workstation 17 Player | Windows Server 2022 | Windows PowerShell 5.1 | Domain: Project.local

**Scripts covered:** Get-AuditReports.ps1 | New-BulkUsers.ps1, Invoke-Offboarding.ps1 and Invoke-Mover.ps1 (tested with deliberately broken CSV files)

---

## 1. Purpose

This document answers two questions about the toolkit:

1. **What does an access review find?** Part A covers the four audit reports and what they found in my lab domain.
2. **What happens when the input data is broken?** Part B covers the tests I ran with deliberately messy CSV files and how the scripts defend themselves.

## Part A: Audit reports

### Why audits matter

In Project 1 I ran a quarterly access review by hand and found three access anomalies. That took a lot of clicking, and it only works if someone remembers to do it. Get-AuditReports.ps1 turns the same checks into one command. It is read-only: it never changes anything in Active Directory.

### The four reports

| Report | What it checks | Why it matters |
|---|---|---|
| 1. Inactive users | Enabled accounts that never logged on, or have not logged on | Unused accounts are an easy target and are often forgotten |

| Report | What it checks | Why it matters |
|---|---|---|
| | for 30 days or more (adjustable with -InactiveDays) | |
| 2. Locked-out accounts | Which accounts are locked out right now | Shows helpdesk work in progress and can point to password-guessing attempts |
| 3. Password expiry | For each enabled account: expired, expiring within 14 days (adjustable with -WarnDays), OK, never expires, or must change at next logon | Expired and never-expiring passwords are both audit findings |
| 4. Group membership audit | For each department group: disabled accounts that are still members, and members who sit outside that department's OU | Catches privilege creep and forgotten leavers |

```
.\Get-AuditReports.ps1
.\Get-AuditReports.ps1 -InactiveDays 7 -WarnDays 30
```

Each report prints on screen and is also saved as a CSV in the Logs folder (a CSV is only written when a report has rows). Each report runs inside its own error handler, so if one fails the other three still run.

## Reports 1 and 2: inactive and locked-out accounts

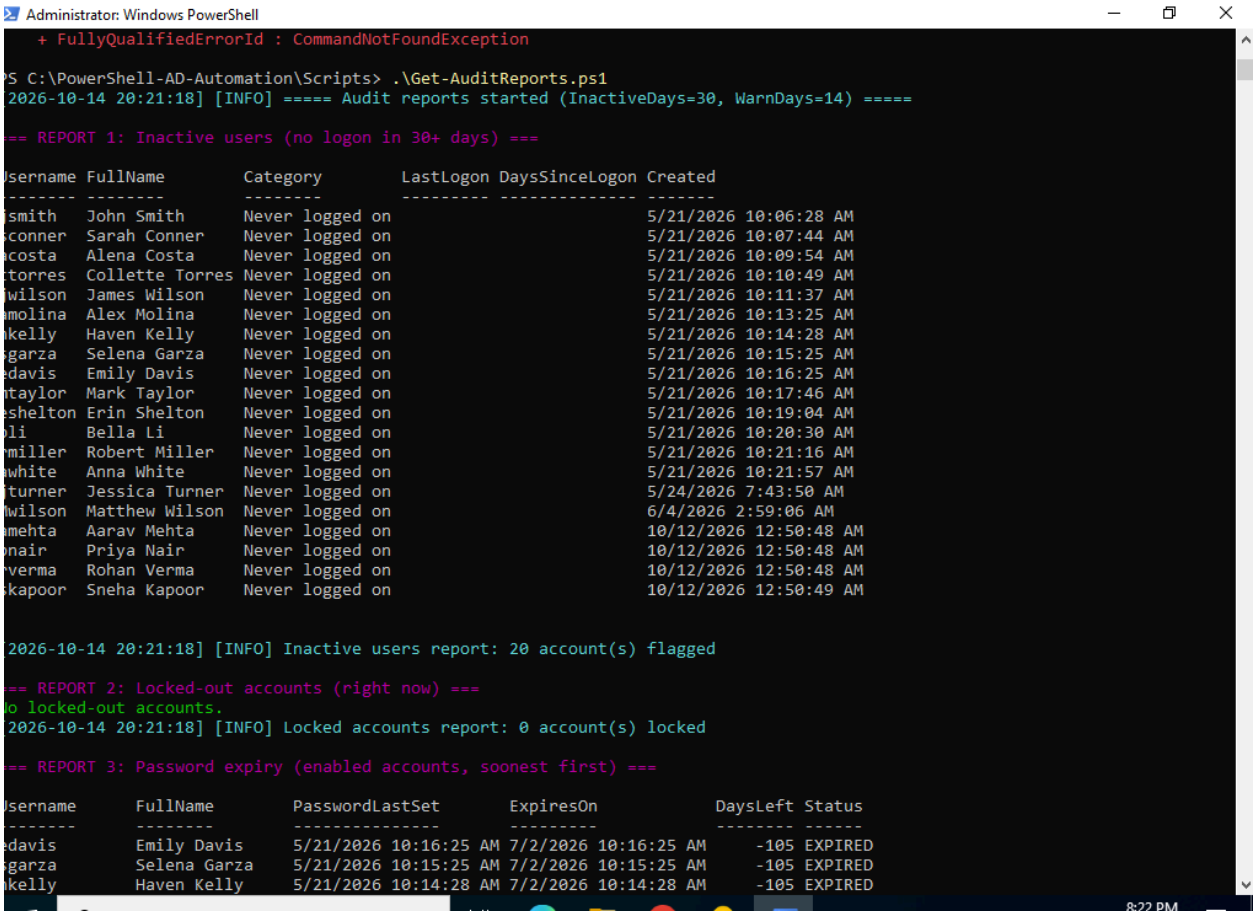



*Figure 1: Audit reports 1 and 2, and the start of report 3.*

Report 1 flagged 20 accounts, and every one of them is listed as Never logged on. That makes sense in a lab: the original lab accounts were created in May 2026 and nobody ever signed in with them, and the new hires I created on 12 October have not signed in either. Report 2 found no locked-out accounts. The first rows of report 3 are visible at the bottom of the screenshot.

One caveat I learned: the last logon date in Active Directory is only updated about every 14 days, so "inactive" is approximate. It is good enough for an access review, but not for real-time monitoring.

## Reports 3 and 4: password expiry and group membership

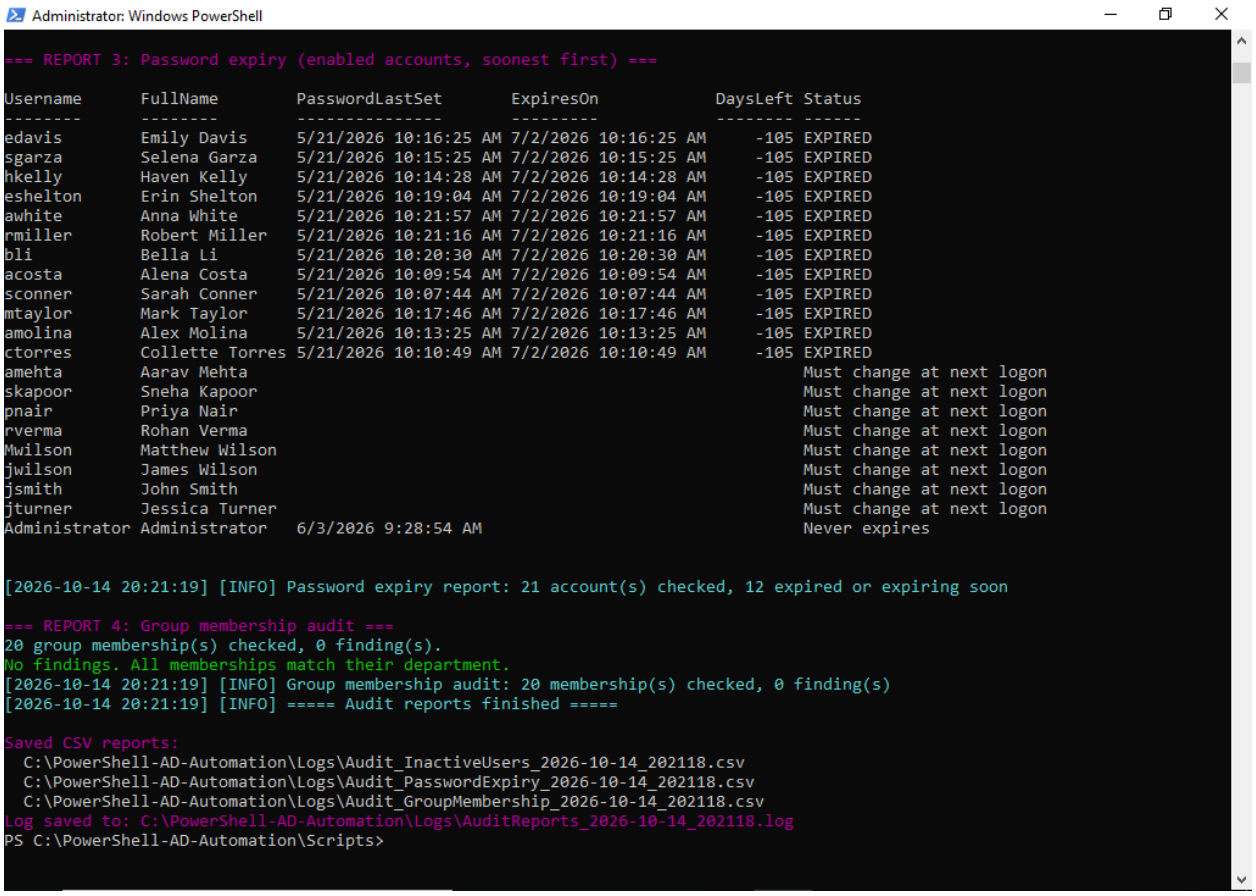



*Figure 2: Password expiry detail and a clean group membership audit with no findings.*

Report 3 checked 21 enabled accounts and found 12 that were expired or expiring soon. The expired ones show -105 days left: their passwords were last set on 21 May 2026 and expired on 2 July 2026, about six weeks later. The new hires show Must change at next logon, because their temporary password has to be replaced at first sign-in. The Administrator account shows Never expires.

Report 4 checked 20 group memberships and found nothing. That is my clean baseline.

## Planting an anomaly

A clean result does not prove the audit can find a problem, so I planted one. I added Sneha Kapoor, who works in Management, to the IT-Team group, ran the audit, and then removed her again:

```
Add-ADGroupMember -Identity "IT-Team" -Members skapoor
.\Get-AuditReports.ps1
```

```
Remove-ADGroupMember -Identity "IT-Team" -Members skapoor
-Confirm:$false
```

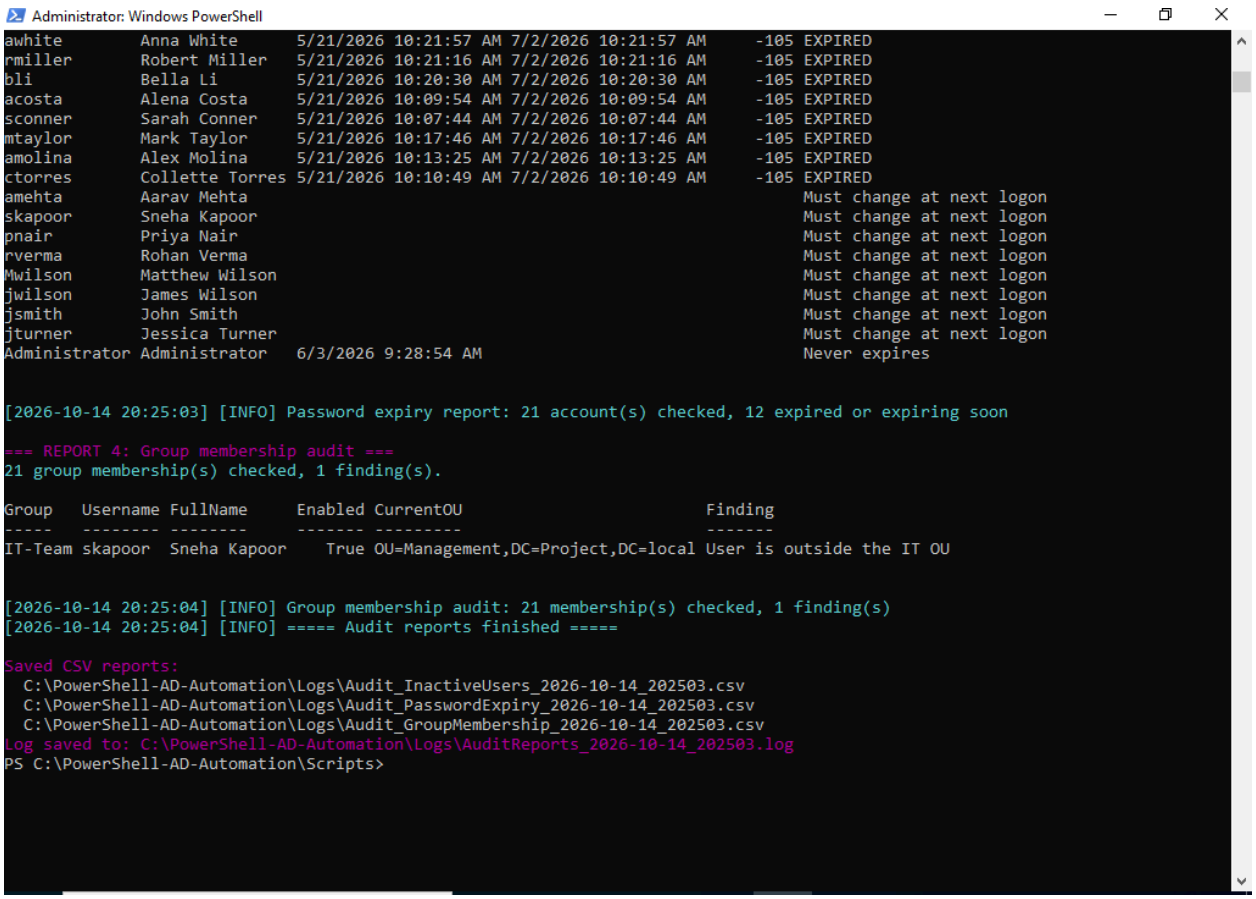



*Figure 3: The group membership audit detects the planted anomaly: a Management user inside IT-Team.*

This time the group audit checked 21 memberships and reported one finding: skapoor is a member of IT-Team but her account sits in the Management OU, with the finding "User is outside the IT OU". This is the same kind of anomaly I tracked down by hand in my Project 1 access review, and now the script finds it automatically. After the test I removed the membership.

## Findings summary

| Report | Result | What it means |
|---|---|---|
| 1. Inactive users | 20 accounts flagged, all Never logged on | Lab and new accounts that have not signed in. In a real company I would |

| Report | Result | What it means |
|---|---|---|
| | | check whether each account is still needed |
| 2. Locked-out accounts | None | No action needed |
| 3. Password expiry | 21 accounts checked, 12 expired or expiring soon | Old lab passwords. In production these would be reset, or the accounts disabled |
| 4. Group membership | 20 memberships and no findings, then 21 memberships and 1 finding after the planted test | The group audit works and detects access that does not match a person's department |

These are lab accounts, so the findings show that the reports work. They are not a real security problem.

## Part B: Validation testing

### Why I tested with bad data

Real HR spreadsheets are never perfect: blank cells, typos, duplicate rows, people who do not exist. My first versions of the scripts only worked on perfect input. In Checkpoint 5 I made them defend themselves, and then I tried to break them.

**What changed in Checkpoint 5:**

- A shared module (Shared-Functions.psm1) replaced duplicated code. The password generator had been copy-pasted into two scripts, and that is risky because a bug fixed in one copy would survive in the other.
- Every script checks its CSV before touching Active Directory: the file exists, is readable, has data rows and has the required columns.
- Every row is checked for missing fields, invalid name characters, unknown departments (any capitalisation is accepted), duplicate rows, and missing OUs or groups.
- If a new hire's username already belongs to a different person, they get a numbered username (amehta2). If it belongs to the same person, the row is skipped.
- If a script fails halfway, the report lists which steps finished.

- Every log starts with who ran the script, on which machine and whether it was a live run or a dry run. Every report gained a Note column that explains each outcome.

### How I tested

I wrote one deliberately messy CSV file for each script and ran each script with -WhatIf, so Active Directory could not be changed. The goal: every bad row is skipped with a reason, the good rows are still processed, and nothing crashes.

### Test 1: onboarding with messy data

```
.\New-BulkUsers.ps1 -CsvPath ..\CSV\NewHires_Messy.csv -WhatIf
```

| Row | Input | Outcome |
|---|---|---|
| 1 | Amit Mehta, IT | Dry run as amehta2, because amehta belongs to another user |
| 2 | Divya Rao, Marketing | Skipped: unknown department |
| 3 | Vikram, last name blank | Skipped: invalid row (Missing: LastName) |
| 4 | Test3 User9 | Skipped: invalid name (digits are not allowed) |
| 5 | Anjali Sharma, department typed as "hr" | Dry run as asharma (lowercase department accepted) |
| 6 | Anjali Sharma, HR | Skipped: duplicate row |
| 7 | Rohan Verma, Finance | Skipped: already exists as rverma |

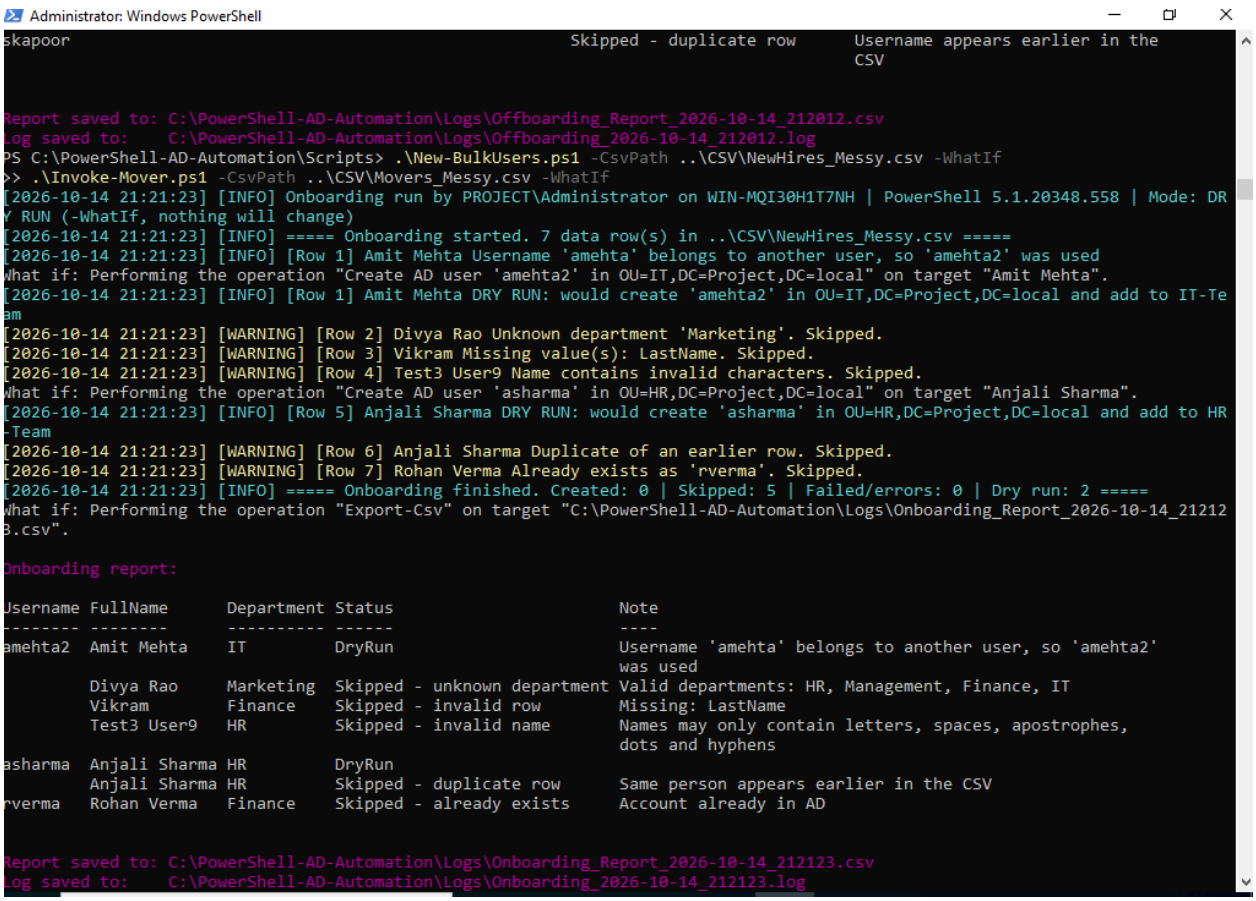



*Figure 4: Onboarding with a messy CSV. Bad rows are skipped with a reason, and the username collision is resolved.*

The summary line reads Created: 0 | Skipped: 5 | Failed/errors: 0 | Dry run: 2. The first log line shows the run header: who ran the script, the computer name, the PowerShell version and the mode, DRY RUN. The Note column in the table explains every outcome in plain English.

### Test 2: offboarding with messy data

```
.\Invoke-Offboarding.ps1 -CsvPath ..\CSV\Leavers_Messy.csv -WhatIf
```

| Row | Input | Outcome |
|---|---|---|
| 1 | ghostuser | Skipped: not found |
| 2 | Administrator | Refused: protected account |
| 3 | Username left blank | Skipped: invalid row |
| 4 | njoshi | Skipped: already offboarded |

| Row | Input | Outcome |
|---|---|---|
| 5 | skapoor | Dry run, no changes made |
| 6 | skapoor again | Skipped: duplicate row |

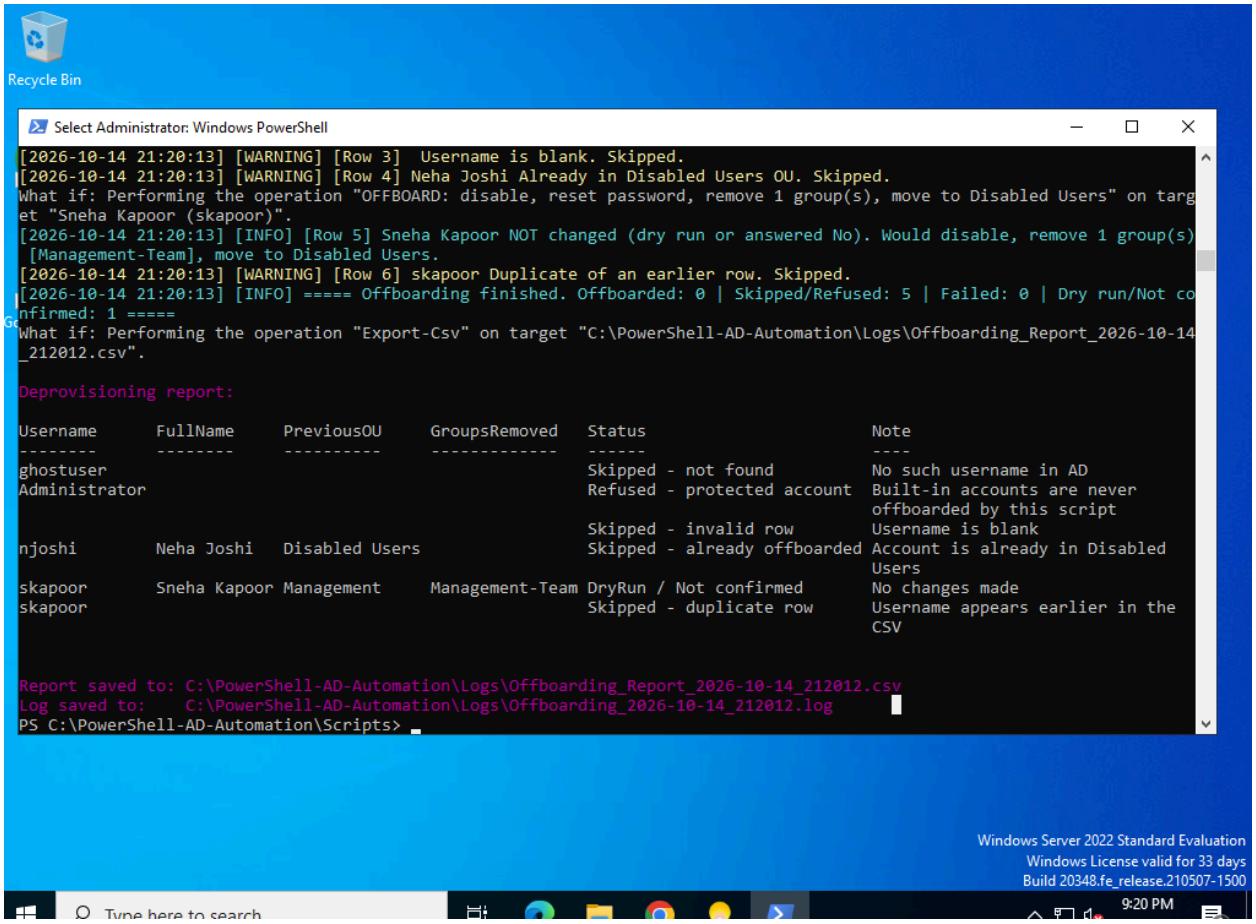



*Figure 5: Offboarding with a messy CSV. The protected account is refused and the other bad rows are skipped.*

The summary reads Offboarded: 0 | Skipped/Refused: 5 | Failed: 0 | Dry run/Not confirmed: 1. Refusing the Administrator account is the most important result here: a typo in a CSV file must never be able to lock out the most powerful account in the domain. The skapoor dry run lists only Management-Team, which also confirms that the group I added for the audit test had been removed.

### Test 3: movers with messy data

```
.\Invoke-Mover.ps1 -CsvPath ..\CSV\Movers_Messy.csv -WhatIf
```

| Row | Input | Outcome |
|---|---|---|
| 1 | ghostuser to Finance | Skipped: not found |
| 2 | njoshi to Finance | Refused: account disabled |
| 3 | amehta to Marketing | Skipped: unknown department |
| 4 | amehta to Management | Skipped: already in department |
| 5 | rverma to "it" (lowercase) | Dry run: Finance to IT, adding IT-Team and removing Finance-Team |

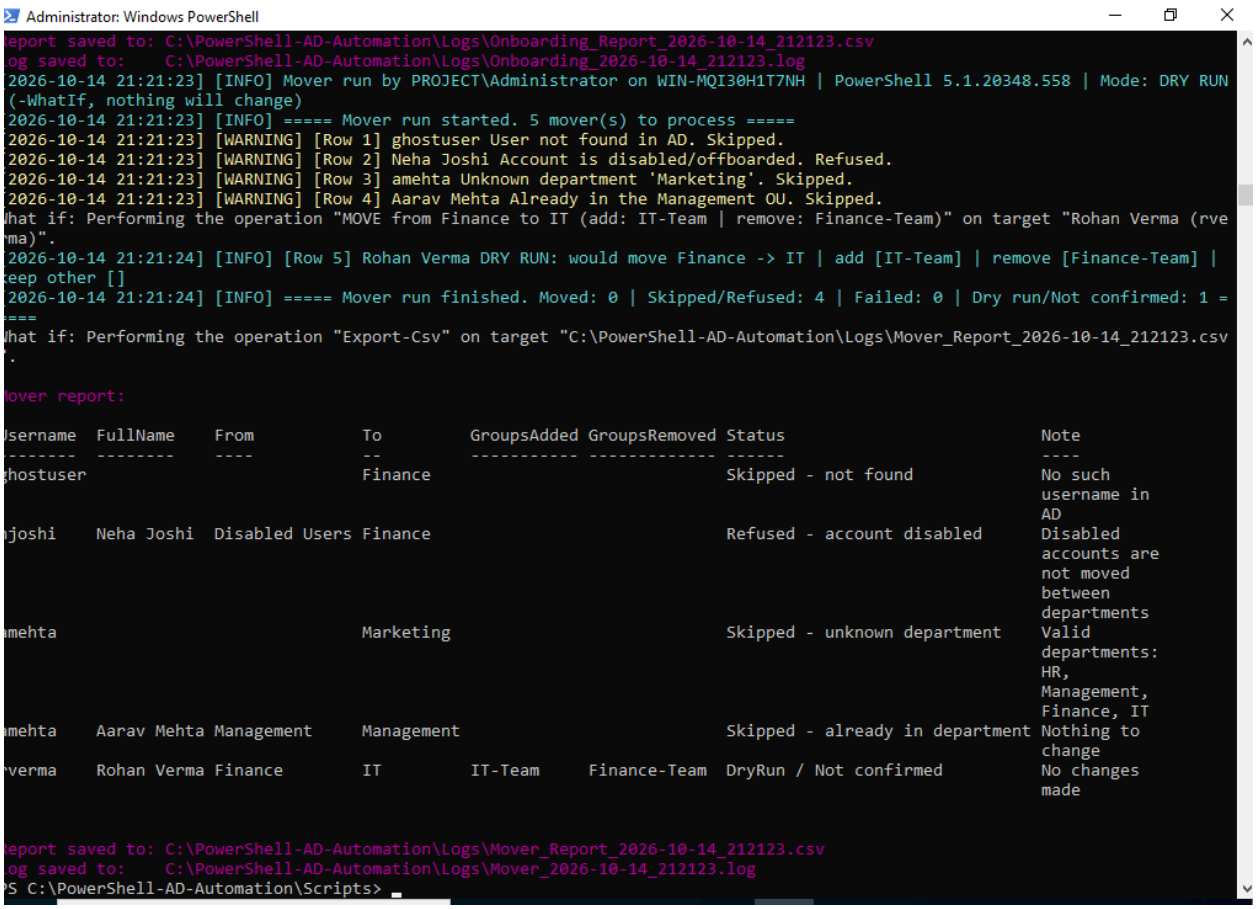



*Figure 6: Movers with a messy CSV. A disabled account is refused and only the valid row produces a plan.*

The summary reads Moved: 0 | Skipped/Refused: 4 | Failed: 0 | Dry run/Not confirmed: 1. Refusing to move a disabled account matters because an offboarded user should never be quietly given new department access.

### Test 4: re-running the original file

After all the changes I ran the original, clean NewHires.csv for real, to be sure the improved script still behaves:

```
.\New-BulkUsers.ps1
```

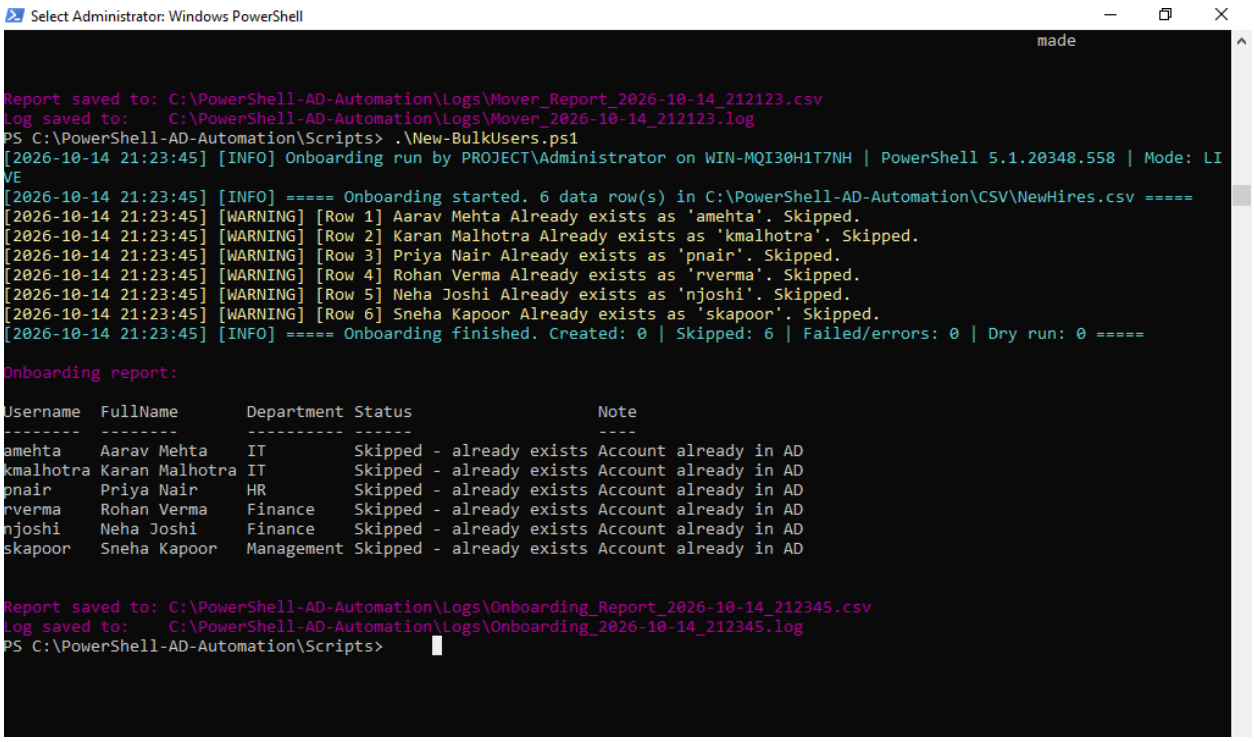



*Figure 7: Re-running onboarding on the original CSV. All six users are skipped because they already exist.*

Every row is skipped with the warning "Already exists" and the Note "Account already in AD". The summary reads Created: 0 | Skipped: 6 | Failed/errors: 0 | Dry run: 0, and the run header shows Mode: LIVE. This proves two things: the script is safe to run twice, and the Checkpoint 5 changes did not break the original behaviour.

## Error-handling design

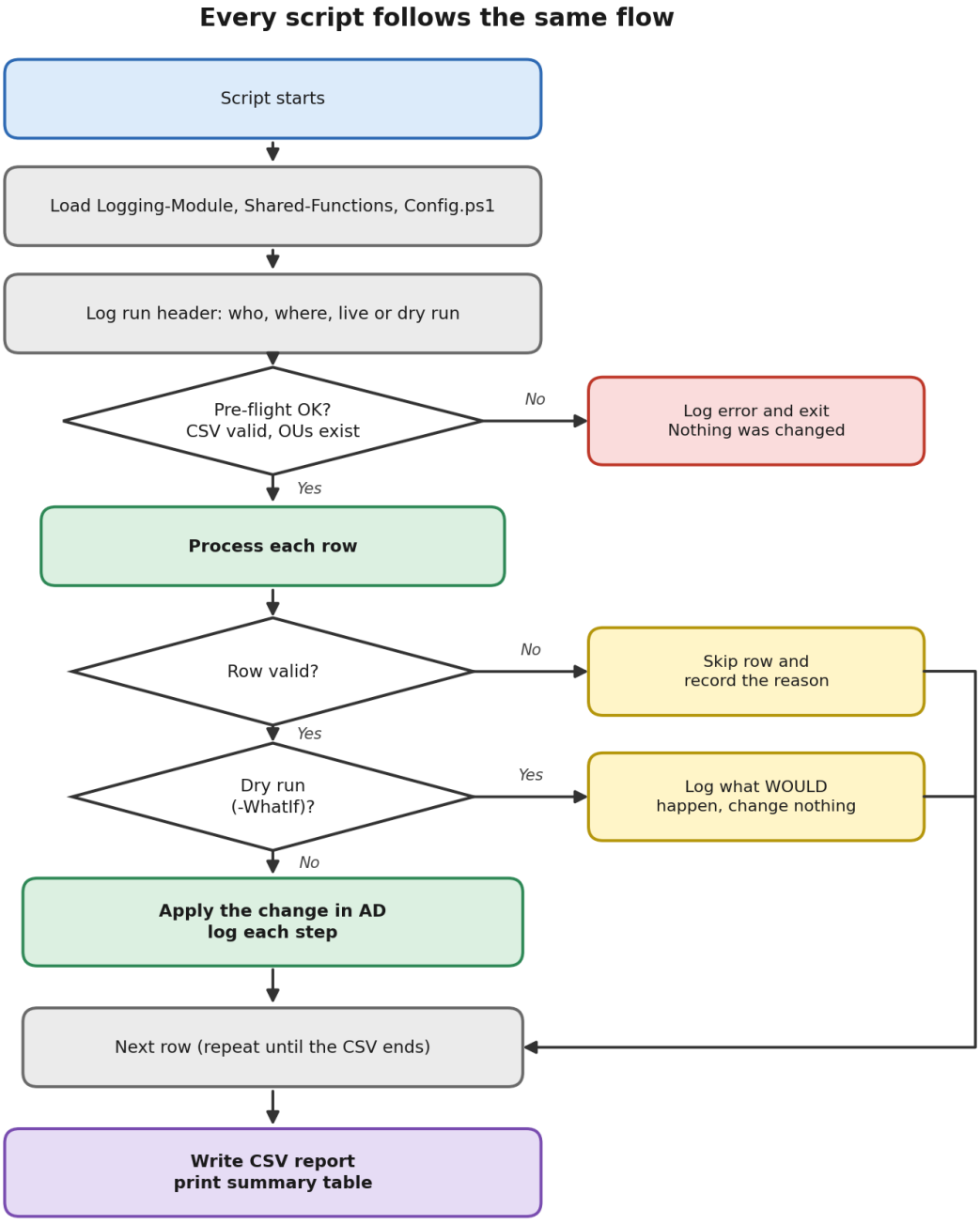



*Figure 8: The flow every script follows: check first, process row by row, skip and explain bad rows, and report at the end.*

- **Fail early and change nothing.** If the CSV is missing or an OU does not exist, the script stops before it touches Active Directory.

- **Isolate bad rows.** One bad row is skipped and explained, and the rest of the file keeps going.
- **Readable errors.** Every change in AD is wrapped in error handling, and common AD errors are turned into plain sentences in the log and report.
- **Report partial failures.** If a multi-step change stops halfway, the report lists the steps that finished. For movers it warns that the user may hold both old and new access.
- **Safe to repeat.** Running a script twice does not duplicate or redo anything.

I want to be clear about one thing: I have not triggered a real mid-run failure in the lab. The partial-failure reporting describes how the scripts are built, not something I have observed.

## Limitations and next steps

- The last logon date updates only about every 14 days, so the inactive-user report is approximate.
- The group audit assumes that a department group belongs with that department's OU, and it does not look inside nested groups.
- Temporary passwords are saved in a plain CSV hand-off file. That is fine in a lab, but a real deployment would use a password vault or one-time links.
- Password generation uses PowerShell's Get-Random, which is not cryptographically secure.
- There is no ticketing or email integration yet, and the ticket number is only recorded.
- There are no automated tests yet. A Pester test suite is a natural next step.
- Next: schedule the audit reports with Task Scheduler, add automated tests, and apply the same ideas to Microsoft 365 in my next lab.

## Lessons

- A clean result proves little. Planting a problem and watching the audit catch it proves much more.
- Breaking my own scripts with bad data taught me more about what they do than running them on good data.
- Dry runs make testing safe, because I can feed the scripts nonsense without risking the directory.